

Learning Concept Hierarchies by Compression

A tutorial note: from Formal Concept Analysis to a probabilistic, minimum-description-length reworking in the spirit of Barlow’s sparse codes

Peter Földiák

July 15, 2026

Abstract

Suppose you are handed a large table of yes/no facts — which animals have which features, which documents contain which words, which patients show which symptoms — and asked: *what are the underlying concepts, and how do they relate?* A classical answer, Formal Concept Analysis (FCA), builds every concept that the data logically permits. The trouble is that real data is noisy, and FCA faithfully invents a concept for every accidental coincidence, producing tens of thousands of “concepts” where a handful were intended. This note develops an alternative built on a single idea from information theory: a concept is worth having exactly when positing it lets us *describe the data more briefly*. We reframe concept learning as *lossy compression with a hierarchy*, connect it to Barlow’s principle that perception should remove redundancy by recruiting a unit for each “suspicious coincidence,” and give a simple greedy algorithm that recovers planted hierarchies while ignoring noise. We report what a first prototype achieves (clean recovery of two- and three-level hierarchies, against an FCA lattice tens of thousands of concepts large), and we end with an honest account of a subtle limitation we uncovered about *overlapping* concepts — one that turns out not to be a bug in the search but a real consequence of how we price the code, and which points directly at the next design decision.

Contents

1	The problem	2
1.1	Binary data and the question of concepts	2
1.2	Why not just cluster, or build a tree?	2
2	Background: two classical answers, and one guiding principle	3
2.1	Formal Concept Analysis, and how it over-generates	3
2.2	Barlow: perception as redundancy reduction	3
2.3	Minimum Description Length: compression as model selection	4
3	A quick information-theory primer	4
4	The model: a concept DAG that generates noisy data	5
4.1	Structure and semantics	5
4.2	Where the randomness lives	5
4.3	The total codelength	6
5	The key identity: “correlation” <i>is</i> change-in-codelength	6
6	The algorithm: grow a DAG by greedy pair-merging	7
6.1	The main loop	7
6.2	One thing we had to add: escaping the “no-noise” trap	8

7	Results so far	8
7.1	Compression trajectory	8
7.2	Recovery quality	8
8	An honest limitation: overlapping concepts	9
8.1	The tempting argument, and the experiment that refuted it	9
8.2	Why a shared child cannot pay rent — the concavity of entropy	9
8.3	The real diagnosis — and it is a feature, not a bug	10
9	Open questions and design decisions	11
	Summary	12

1 The problem

1.1 Binary data and the question of concepts

Much of the world’s data can be written as a grid of yes/no answers. Rows are *objects* (animals, documents, customers, cells); columns are *attributes* (features, words, purchases, active genes). A cell is 1 if the object has the attribute and 0 otherwise. We call this an object $\times$ attribute *context*, and write it as a matrix X with N rows and A columns.

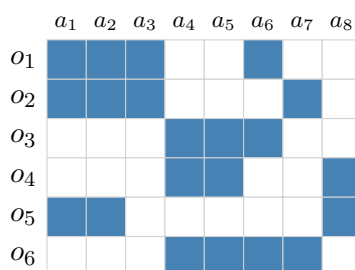


Figure 1: A tiny binary context X ($N = 6$ objects, $A = 8$ attributes). A filled cell is a 1. The eye already picks out “blocks”: objects o_1, o_2 share a_1, a_2, a_3 ; objects o_3, o_4, o_6 share a_4, a_5 . Those blocks are the kind of latent *concept* we want a machine to discover — but only the real ones, not every accidental overlap.

Given only X , we would like to recover a small set of *concepts* — groups of attributes that tend to occur together for a reason — and the *is-a* relationships among them (a *penguin* is a *bird* is an *animal*). Concretely we want two things:

1. a compact vocabulary of concepts, each identified with the set of attributes it implies (its *intent*), and
2. a partial order (a hierarchy) saying which concepts specialise which.

This is unsupervised structure learning: nobody labels the concepts; they must be inferred from the pattern of co-occurrence alone.

1.2 Why not just cluster, or build a tree?

Two familiar tools fall short in instructive ways.

Clustering assigns each object to one group. But objects genuinely belong to several concepts at once: a tomato is a *fruit* and *red* and *round*. Forcing one label per object destroys exactly the combinatorial structure we are after.

Hierarchical clustering / taxonomy trees allow nesting but insist that every concept has a single parent. Real concept systems are not trees: “red” and “round” are two independent axes, and “red round thing” sits under *both*. A structure that allows multiple parents is a *directed acyclic graph* (DAG), not a tree. Keeping the DAG generality — and learning it from data — is a central goal of this project.

2 Background: two classical answers, and one guiding principle

2.1 Formal Concept Analysis, and how it over-generates

Formal Concept Analysis [8] gives a beautiful, exact answer to “what are all the concepts?”. Given the context X , a *formal concept* is a pair (set of objects, set of attributes) that is *closed*: the objects are exactly those having all the attributes, and the attributes are exactly those shared by all the objects. Order the concepts by inclusion of their object sets and you get the *concept lattice* — a complete, canonical map of every implication in the data.

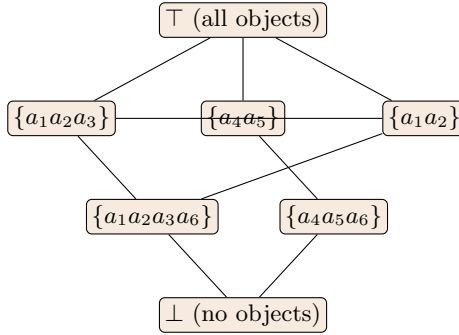


Figure 2: A fragment of a concept lattice for data like Figure 1. Each node is a closed attribute set. On clean data this is exactly the structure we want. The problem is what happens under noise (next).

The catch is *closure under intersection*. Every intersection of object sets is itself a concept, so the number of concepts can be exponential in the data size. Worse, this is fragile to noise. Flip a single cell — one bird that, by observation error, appears not to fly — and FCA dutifully creates a new concept “birds that fly except this one.” On real data with a few percent noise, the lattice explodes into tens of thousands of concepts, almost all of them accidents. FCA answers “what *could* be a concept?” when we wanted “what *is* a concept?”. It has no notion that a concept should have to *earn its place*.

2.2 Barlow: perception as redundancy reduction

The guiding intuition for a better answer comes from neuroscience. Horace Barlow argued [2, 4] that a central job of sensory coding is to *remove redundancy*: the raw signal is full of correlations (neighbouring pixels are similar, features co-occur), and an efficient brain should re-code it so that the resulting features are as *independent* as possible — a *factorial code*. In such a code, each recurring, statistically *suspicious* coincidence in the input is captured by a dedicated unit, so that downstream the coincidence no longer has to be re-discovered every time.

Barlow’s memorable phrasing is that the visual system should treat a “*suspicious coincidence*” — an event far more common than chance would predict — as evidence for a new feature worth representing. A closely related idea is that good sensory codes are *sparse*: at any moment only a few of many possible units are active [3, 6, 7, 13]. Sparse and factorial codes are two sides of

one coin: if you insist that few units fire and that they are informative, you are pushed towards units that stand for genuine, reusable regularities.

This note takes Barlow’s slogan literally and makes it constructive and exact: *introduce a new concept precisely when a coincidence is suspicious enough to pay for its own description*. To make “pay for its description” precise we need one more ingredient.

2.3 Minimum Description Length: compression as model selection

The Minimum Description Length principle [14, 9] says: *the best model of some data is the one that lets you describe the data in the fewest bits, counting the description of the model itself*. If you must transmit X to a friend, you may first send a *model* (a vocabulary of concepts and a hierarchy) and then send X *using* that model. A good model makes the second part short; a complicated model makes the first part long. MDL picks the total-bits minimum, and this automatically penalises over-fitting: a concept invented for one noisy coincidence costs more to describe than it saves, so it is rejected. No significance threshold or p -value is needed — *the model’s own description cost is the threshold*.

MDL has a very successful incarnation in pattern mining. KRIMP [20] and its greedy successor SLIM [18] learn a *code table*: a flat dictionary of itemsets that together compress a transaction database. Their empirical lesson is encouraging and we lean on it heavily: greedily considering *pairwise unions of current dictionary entries*, judged by exact change in description length, gets remarkably close to what far more expensive global search finds.

The one-sentence thesis of this project. *Take SLIM’s “grow a dictionary by compression” recipe, but replace the flat code table with a DAG of concepts with is-a (closure) semantics; the result is a probabilistic, MDL version of Formal Concept Analysis that only posits a concept when it pays rent — which is Barlow’s suspicious-coincidence unit recruitment made exact. As far as we know the combination (DAG-structured closure dictionary learned by MDL) is new.*

3 A quick information-theory primer

Everything hinges on measuring bits, so here is the minimum needed, from scratch.

Codelength of an event. If an event has probability p , the cheapest possible code spends $-\log_2 p$ bits on it [17]. Rare events are expensive, certain events are free. The *entropy* $H(p) = -p \log_2 p - (1-p) \log_2 (1-p)$ is the average cost per symbol of the best code for a coin that lands heads with probability p . It is 0 for a certain coin ($p \in \{0, 1\}$) and maximal (1 bit) for a fair coin ($p = \frac{1}{2}$). We will need one fact about H : it is *concave* and peaks in the middle. Splitting one column of 1s into two moderate columns can cost *more* total entropy than the single column did — a fact that will return with force in Section 8.

Coding without knowing p in advance. We rarely know the true frequency of a feature. A clean fix is the *Krichevsky–Trofimov (KT) estimator* [10]: to code a binary sequence with n_1 ones and n_0 zeros, predict the next bit with a running estimate that starts at one-half and updates as

$$\Pr(\text{next} = 1) = \frac{n_1 + \frac{1}{2}}{n_1 + n_0 + 1}.$$

The total bits to send the whole sequence has the closed form

$$L_{\text{KT}}(n_1, n_0) = -\log_2 \frac{\Gamma(n_1 + \frac{1}{2}) \Gamma(n_0 + \frac{1}{2})}{\Gamma(\frac{1}{2})^2 \Gamma(n_1 + n_0 + 1)}.$$

You do not need to memorise this; the point is that it is (i) parameter-free — the “ $+\frac{1}{2}$ ” is the only choice, and it is the principled one (a Jeffreys/Bayes prior); and (ii) it depends only on *counts*, so we can evaluate it instantly from tallies. Because KT is a Bayesian mixture code, MDL built on it coincides with Bayesian model selection — a bridge we mention again at the end.

Coding a whole number. To describe the model we sometimes must send an integer k (e.g. “there are k concepts”) without an agreed upper bound. Rissanen’s universal code for integers [15] does this in about $\log_2 k + \log_2 \log_2 k + \dots$ bits — essentially “the number of digits, plus the digits.” Again the detail does not matter; it is simply the honest price of naming a concept count.

4 The model: a concept DAG that generates noisy data

4.1 Structure and semantics

Our model is a DAG with two kinds of nodes (Figure 3):

- **Attribute nodes** — the A observed columns. They are the *sinks* (bottom) of the DAG.
- **Concept nodes** — latent groupings. Each concept points to its *children*, which may be attributes or other (lower) concepts.

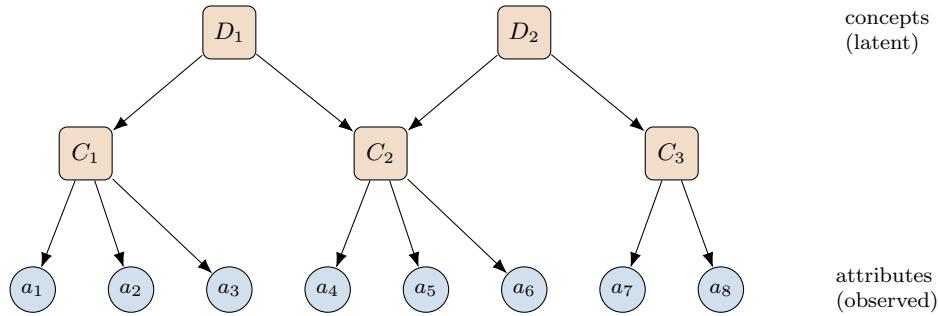


Figure 3: A concept DAG. Activating concept D_1 turns on everything reachable below it: C_1, C_2 and hence attributes $a_1 - a_6$. Note C_2 has *two* parents D_1, D_2 — the multiple-parent structure a tree cannot express. (This particular “shared child” will be the star of Section 8.)

The semantics are *deterministic downward closure*. Each object n comes with an *activation set* S_n of nodes it switches on. Turning on a node turns on everything reachable from it. The predicted attribute vector $\hat{Y}_n$ is the union of all attributes reachable from S_n . Two modelling choices keep this clean:

- S_n is an *antichain*: we never explicitly list a node and also something above it (that would be redundant, since the higher node already implies the lower).
- Objects may activate a *bare attribute* directly. This gives idiosyncratic one-off features a cheap home, so the model is not tempted to invent a junk concept for them.

4.2 Where the randomness lives

Real data is noisy, so the model includes an *observation noise* layer with two rates:

$$\varepsilon^+ = \Pr(\text{observe } 1 \mid \text{generated } 0) \text{ (false positive),} \quad \varepsilon^- = \Pr(\text{observe } 0 \mid \text{generated } 1) \text{ (false negative).}$$

The story for one object is thus: pick an antichain S_n ; propagate closure to get the clean prediction $\hat{Y}_n$; then flip each cell independently with the appropriate small probability to get the observed row X_n (Figure 4). All stochasticity sits at this final layer; inside the DAG, closure is exact. (Letting activation itself be probabilistic — a “noisy-OR” belief network — is a deliberate future extension, discussed at the end.)

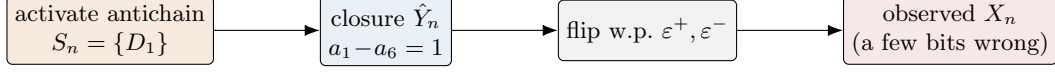


Figure 4: The generative process for a single object. Learning inverts this: from many observed rows X_n , recover the DAG and the (hidden) activations.

4.3 The total codelength

The description we must minimise has three parts:

$$L(G, X) = \underbrace{L(G)}_{\text{structure}} + \underbrace{\sum_n L(S_n | G)}_{\text{usage}} + \underbrace{\sum_n L(X_n | S_n, G)}_{\text{residual}}. \quad (1)$$

Structure $L(G)$. The price of the map itself: how many concepts (a Rissanen integer code), and which edges exist (coded with a single global KT Bernoulli, so a dense DAG makes each edge cheap and a sparse one makes each edge dear — adaptive, nothing hand-tuned).

Usage $L(S_n | G)$ — **the rent**. For each dictionary item k (concept or bare attribute) we code the column “is k explicitly active in object n ?” across all objects, with a per-item KT Bernoulli. *This is the term that makes concepts pay rent*. A concept used by many objects gets a cheap per-use price and is a bargain; a concept used once costs almost as much as spelling out its attributes raw, so it dies. This is precisely the mechanism FCA lacks.

Residual $L(X_n | S_n, G)$. The corrections: the false positives and false negatives between the prediction $\hat{Y}_n$ and the observation X_n , coded with KT-estimated ϵ^+, ϵ^- .

MDL now selects the DAG minimising the sum (1). Three degenerate extremes are automatically punished, which is a good sanity check:

Model	structure $L(G)$	residual
Full FCA lattice (a concept per coincidence)	enormous	zero
One concept per distinct object	enormous	zero
Empty model (no concepts)	zero	enormous

The good ontology lives in the valley between them.

5 The key identity: “correlation” *is* change-in-codelength

Here is the bridge from Barlow’s slogan to a concrete algorithm. Suppose we are considering creating a concept c that stands for the co-occurrence of two existing dictionary items a and b . Let n_{ab} be the number of objects that currently use *both*. Introducing c and rewriting those objects to use c instead of $a+b$ changes the total codelength by approximately

$$\Delta L \approx -n_{ab} \left[\underbrace{-\log_2 \hat{q}_a - \log_2 \hat{q}_b + \log_2 \hat{q}_c}_{\text{bits saved per shared use}} \right] + \text{overhead}, \quad (2)$$

where $\hat{q}_\bullet$ are the KT-estimated usage frequencies and the overhead is the structure bits for the new node and its edges, plus the new usage column, *plus a subtle re-pricing of a and b whose own usage frequencies drop once c absorbs their joint occurrences*. To first order this is

$$\Delta L \approx -N \cdot I(a;b) + \text{overhead},$$

where $I(a;b)$ is the *mutual information* between the two items: their statistical dependence, weighted by how much data we have. In words:

The number of bits a new concept saves is the strength of the coincidence it captures, times the amount of evidence for it, minus the cost of writing the concept down.

This is exactly Barlow’s “suspicious coincidence,” quantified. “Add the strongest correlation” and “take the most compressive MDL step” are the *same move*; the MDL form simply gets the acceptance threshold and the tie-breaking right for free, and — crucially — includes the re-pricing term that naïve correlation scores miss.

6 The algorithm: grow a DAG by greedy pair-merging

6.1 The main loop

We start from the simplest possible model — no concepts, every object coded by its raw attributes — and repeatedly add the single most profitable concept.

```

initialise: dictionary = attributes only; each object's code = its 1s
repeat:
    rank candidate pairs (a,b) by a cheap upper bound on their saving
        (bound = n_ab * pointwise-mutual-information)
    take the best candidate; compute its EXACT change-in-codelength dL
        (evaluate lazily: only re-check the top few, since gains
         almost only shrink as the dictionary grows -- a safe shortcut)
    if dL < 0 (it compresses):
        create concept c = {a,b}
        re-encode ONLY the objects that used both a and b (use an index)
        update the counts
    every few accepts: PRUNE-AND-COLLAPSE weak or redundant concepts
until no candidate compresses

```

Three points deserve unpacking.

Everything is a count. The score (1) and every ΔL depend only on tallies: how often each item is used, how often each pair is co-used, and the residual error totals. We keep these in a *counter store* and the scorer talks *only* to it, never re-scanning the raw data. This is what makes each candidate evaluation $O(1)$ instead of $O(N)$, and it is the design commitment that later lets the same code run in a streaming/online mode.

Explaining away comes for free. We track co-occurrence in the *current code* S_n , not in the raw data. The moment $c = \{a, b\}$ is created and absorbs their joint uses, the residual co-occurrence of a and b collapses, so we never re-propose it — and *new* correlations involving c itself become visible. That is precisely how the hierarchy climbs: pairs of base concepts become the children of a mid concept, pairs of mids become the children of a top.

Devaluation and the pruning sweep. A five-attribute concept gets built as a *ladder* of binary merges, and the intermediate rungs are often used *only* inside the bigger concept above them. A periodic sweep asks of each concept: if I delete it and re-route its uses to its children, does the total shrink? If so, collapse it. Without this the dictionary silts up with dead rungs.

6.2 One thing we had to add: escaping the “no-noise” trap

We found, and fixed, a subtle failure. Early in learning the model is perfect on the (still-uncompressed) data — zero residual error. Under the KT code, the *first* false negative is very expensive (its estimated rate is near zero, so admitting one costs ~ 15 bits), so no single object will ever volunteer to adopt a concept-plus-noise encoding, even though a whole *cohort* of similar objects doing so together is cheap. The learner gets stuck coding noise via junk concepts. Our fix is a *batch re-encoding* step: periodically re-encode *all* objects at once using a few trial noise rates, and accept the whole pass only if the exact global codelength drops. The trial rates are just a proposal device; acceptance is still the one true score, so no threshold sneaks into the model. This is worth flagging because it will recur in any MDL learner with a costly “first error”: the escape has to be collective, not per-object.

7 Results so far

We validate on *planted* data: we build a known hierarchy, sample objects from it with 2% noise, hand the learner only X , and ask whether it recovers the hierarchy and beats the description length of the very model that generated the data.

7.1 Compression trajectory

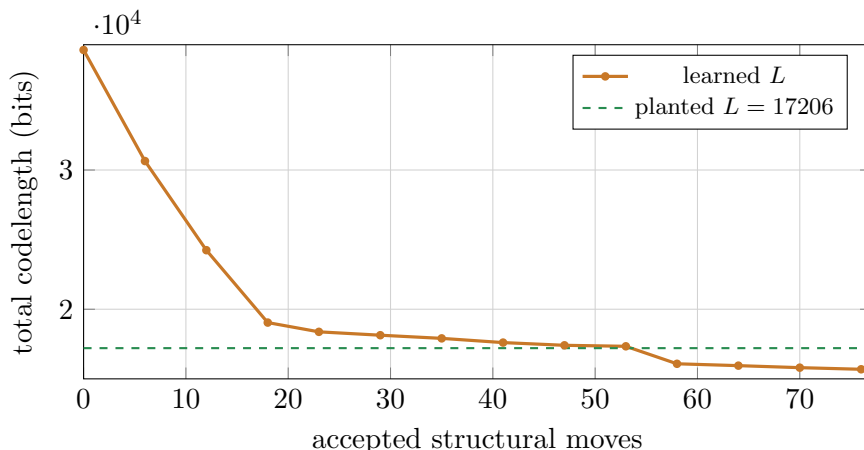


Figure 5: The learner’s total codelength as it adds concepts, on a planted two-level hierarchy ($A = 20$ attributes, 6 concepts, $N = 2000$ objects, 2% noise). Every accepted move provably decreases the score (we assert this). The final model is *shorter* than the planted one (green line): having seen the noise, the learner codes it slightly more efficiently than the generating model does on its own sample.

7.2 Recovery quality

The headline is the last two columns of Table 1 read together: where classical FCA reports *tens of thousands* of concepts on this noisy data, our method reports the *six* (or fourteen) that pay rent, recovers them essentially exactly, and does so while describing the data in fewer bits than the true generator uses. The three-level result matters especially: it shows that depth *builds on depth* — mid-level concepts are discovered from the co-occurrence of already-learned base concepts, and top concepts from the co-occurrence of already-learned mids — using nothing but repeated pairwise merging.

planted hierarchy	concepts recovered	extent Jaccard	order P/R	L ratio	FCA lattice size
2 levels ($A=20, K=6$)	6/6	≥ 0.99	1.00/1.00	0.91	57 000+
3 levels ($A=30, K=14$)	14/14	≥ 0.96	1.00/1.00	0.92	(explodes)

Table 1: Planted-recovery results. *Extent Jaccard*: overlap between each planted concept’s set of objects and its best-matching learned concept (1.0 = identical). *Order P/R*: precision and recall of the learned is-a relation against the planted partial order. *L ratio*: learned / planted codelength (< 1 = the learner compresses better than the generator). *FCA lattice size*: number of closed sets a classical FCA would report on the same noisy data — the “what we saved you from” number.

8 An honest limitation: overlapping concepts

Here is where the story gets interesting, and where we spent this round of work. All the successes above share a feature: the planted concepts have *disjoint* children. Each attribute belongs to one base concept; each base belongs to one parent. What happens when concepts genuinely *overlap* — when one concept is a child of two different parents (the shared C_2 in Figure 3)? A tomato is both a fruit and a red-round-thing; “edible” might sit under both. Overlap is the whole reason we wanted a DAG rather than a tree, so this is not a corner case — it is the point.

8.1 The tempting argument, and the experiment that refuted it

One hopes overlap simply “drops out” of the principles: a shared child is just a coincidence used by two parents; surely compression handles it. We tested this directly. We planted the DAG of Figure 3 — two top concepts $D_1 = \{C_1, C_2\}$ and $D_2 = \{C_2, C_3\}$ sharing the middle base C_2 — sampled 5000 noisy objects with the tops as the dominant generators, and ran the *unchanged* learner.

The result was sharp and, at first, disappointing:

- The three base concepts C_1, C_2, C_3 were recovered *perfectly* (Jaccard 1.00).
- *Neither* top concept was recovered.
- The learner’s codelength was *far below* the planted model’s (ratio ≈ 0.87).

That last point is the tell. The learner is not failing to search — it is *correctly* finding that, under our current score, the flat three-base model is genuinely cheaper than the intended five-node hierarchy. When we asked the scorer directly what it would cost to add the true top $D_1 = \{C_1, C_2\}$ at the learned solution, the answer was **+3051 bits** — a large *increase* — despite C_1 and C_2 co-occurring 1607 times out of 5000. Why?

8.2 Why a shared child cannot pay rent — the concavity of entropy

Recall that with deterministic closure, an intermediate concept changes *nothing* about the prediction (the base concepts already predict their attributes exactly). So the residual term does not move at all. The *only* effect of adding D_1 is on the *usage* columns — and there it is forced to turn *two* columns into *three*:

Now use the concavity of entropy. Writing u/N for a column’s density, its cost is about $N \cdot H(u/N)$ bits. In the shared experiment the numbers were:

before: $N[H(0.45)+H(0.73)] \approx 9160$ bits, after: $N[H(0.13)+H(0.41)+H(0.32)] \approx 12190$ bits.

The total number of 1s actually went *down* (the shared uses are no longer double-counted), yet the cost went *up* by ~ 3000 bits, because splitting into a third moderate-density column adds

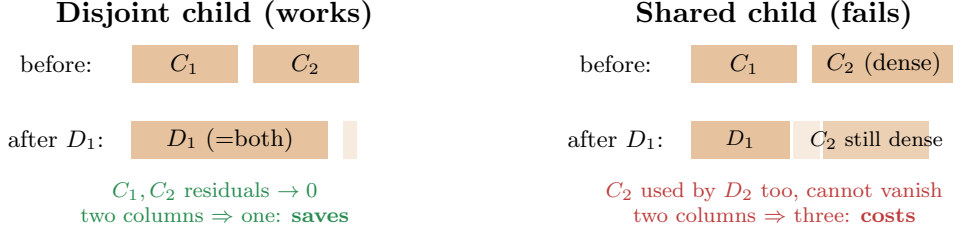


Figure 6: Why disjoint concepts pay rent and shared ones do not, in terms of usage columns (width $\propto$ how many objects explicitly use the item). *Left*: when C_1, C_2 belong only to D_1 , creating D_1 empties their columns — two columns become one. *Right*: when C_2 is shared with another parent D_2 , its column stays dense no matter what D_1 does — so we have simply *added* a third column.

entropy faster than the removed 1s save it. Figure 7 shows the geometry: three points partway up the hump sum higher than two.

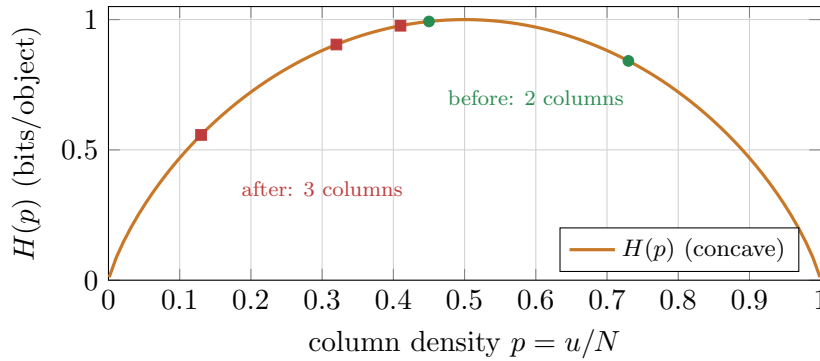


Figure 7: Entropy is concave. Two moderately dense columns (green circles, sum ≈ 1.83) become three columns of middling density (red squares, sum ≈ 2.44) when a shared child is factored out — *more* total cost even though fewer 1s are stored. This, not a search failure, is why the current model refuses overlapping concepts.

By contrast, when the child is *not* shared (Figure 6, left) its residual column drops to zero and two columns genuinely become one — so the disjoint hierarchies of the previous section pay rent easily. We confirmed this too: the same experiment with disjoint parents recovers *all* concepts perfectly.

8.3 The real diagnosis — and it is a feature, not a bug

The mistake in the tempting argument was to treat the usage columns as *independent*. They are not: an object uses D_1 , *or* it uses C_1 and C_2 separately, but rarely both — the parent and its children’s residual columns are strongly *anti*-correlated. Our per-item code prices each column as if the items were independent Bernoulli variables, and so it *double pays* whenever they are dependent. For disjoint structure the closure DAG can actually make the explicit activations near-independent (that is what compression is doing), and the independent-column price is right. For overlapping structure, no pure closure code can make the activations independent, and the independent-column price penalises the sharing that a DAG is supposed to exploit.

This is a genuinely satisfying diagnosis because it lands us right back on Barlow. The ideal code is *factorial*: its units fire independently. Our usage term is only *correct* when we have reached that ideal. Tree-like (disjoint) hierarchies can reach it; overlapping ones cannot,

under closure semantics with an independent-column code. The learner refusing an overlapping concept is the score honestly reporting “this concept does not make your code more factorial.”

9 Open questions and design decisions

The overlap finding sharpens the central open question rather than blocking progress. In rough priority order:

1. How should usage be coded so that sharing can pay? This is the decision the rest depends on. Three candidate directions:

- **Conditional (DAG-aware) usage coding.** Code a node’s activation *conditional on its parents*: $\Pr(\text{child active} \mid \text{parent active})$. This captures the anti-correlation exactly and should let a shared child pay rent. It is a small change to the counter store (condition the tallies on parent state) but it complicates the acyclic bookkeeping.
- **Noisy-OR propagation** (roadmap item, see below): replace deterministic closure with probabilistic activation down the edges. A shared child then has a real generative role and the independence assumption is relaxed at the source. This is the most principled and the most work.
- **Accept the scope.** Declare that v1 learns the “tree-like backbone” of a concept system and treat overlap as a second-stage refinement. Cheapest, least satisfying, but a legitimate staging.

Our current lean is towards conditional usage coding as the smallest change that restores the principle, with noisy-OR as the eventual home.

2. Richer proposals than pair-merge. Pairwise merging is blind to purely higher-order structure (dependencies with no pairwise signal — a non-issue for is-a data, but worth stating). Planned additions: an *add-edge* move ($k \rightarrow j$ when j is active in most objects using k), which refactors a shared attribute *upward* into a parent and is the natural way to *discover* overlap rather than merely price it; *split* and *merge* moves; and mining candidate concepts from the closed itemsets of the current *residual*.

3. Online / streaming version. Because the score is entirely count-based, the same machinery runs on a stream: decay the counts, update per object, never look back. This is *prequential* MDL [5] and, in neural terms, a Hebbian coincidence detector with an MDL recruitment threshold — arguably the most faithful computational reading of Barlow’s proposal.

4. Bayesian version, nearly for free. Because KT codes are Bayesian mixtures, our MDL score is already approximately a negative log marginal-likelihood. Running Metropolis at temperature 1 over the same proposals samples an approximate *posterior* over DAGs — useful for reporting uncertainty in the recovered structure.

5. Deep, fully generative version. The long-term target replaces deterministic closure with stacked noisy-OR layers and an unbounded nonparametric structure prior (a cascading Indian Buffet Process [1]; relatives include multiple-cause models [16]). Whether learned codes *stay* downward-closed under free propagation is an empirical question we intend to measure rather than assume.

6. Baselines and real data. For a proper evaluation we will compare against flat MDL pattern miners (KRIMP/SLIM [20, 18]), Boolean matrix factorisation with MDL model selection [11, 12], and the correlation-explanation approach CorEx [19], and move from planted data to taxonomy-annotated real contexts — and eventually to binarised sparse neural codes, closing the loop to the original motivation: reading a learned sparse representation *as* an ontology.

Summary

We set out to learn concept hierarchies from noisy binary data without FCA’s combinatorial explosion. The organising idea is old and simple — Barlow’s suspicious coincidence — made exact through minimum description length: posit a concept exactly when it shortens the total description, evaluate that saving $O(1)$ from counts, and let the model cost be the only threshold. A greedy pair-merging learner recovers planted two- and three-level hierarchies cleanly, compressing better than the generating model where classical FCA would report tens of thousands of spurious concepts. Investigating overlapping concepts turned up a clean, principled limitation — an independent-column usage code cannot reward a shared child, because entropy is concave and the shared child’s column never empties — which is not a search failure but the score correctly reporting that overlap does not make the code more factorial. That diagnosis names the next design decision precisely: teach the usage code about the DAG’s own dependence, and the sharing a hierarchy exists to exploit will finally pay its rent.

References

- [1] Ryan P. Adams, Hanna M. Wallach, and Zoubin Ghahramani. Learning the structure of deep sparse graphical models. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 1–8, 2010.
- [2] Horace B. Barlow. Possible principles underlying the transformations of sensory messages. In Walter A. Rosenblith, editor, *Sensory Communication*, pages 217–234. MIT Press, 1961.
- [3] Horace B. Barlow. Single units and sensation: A neuron doctrine for perceptual psychology? *Perception*, 1(4):371–394, 1972.
- [4] Horace B. Barlow. Unsupervised learning. *Neural Computation*, 1(3):295–311, 1989.
- [5] A. Philip Dawid. Present position and potential developments: Some personal views. statistical theory: The prequential approach. *Journal of the Royal Statistical Society. Series A*, 147(2):278–292, 1984.
- [6] Peter Földiák. Forming sparse representations by local anti-Hebbian learning. *Biological Cybernetics*, 64(2):165–170, 1990.
- [7] Peter Földiák and Malcolm P. Young. Sparse coding in the primate cortex. In Michael A. Arbib, editor, *The Handbook of Brain Theory and Neural Networks*, pages 895–898. MIT Press, 1995.
- [8] Bernhard Ganter and Rudolf Wille. *Formal Concept Analysis: Mathematical Foundations*. Springer, 1999.
- [9] Peter D. Grünwald. *The Minimum Description Length Principle*. MIT Press, 2007.
- [10] Raphael E. Krichevsky and Victor K. Trofimov. The performance of universal encoding. *IEEE Transactions on Information Theory*, 27(2):199–207, 1981.

- [11] Pauli Miettinen, Taneli Mielikäinen, Aristides Gionis, Gautam Das, and Heikki Mannila. The discrete basis problem. *IEEE Transactions on Knowledge and Data Engineering*, 20(10):1348–1362, 2008.
- [12] Pauli Miettinen and Jilles Vreeken. Model order selection for boolean matrix factorization. In *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 51–59, 2011.
- [13] Bruno A. Olshausen and David J. Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 381(6583):607–609, 1996.
- [14] Jorma Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978.
- [15] Jorma Rissanen. A universal prior for integers and estimation by minimum description length. *The Annals of Statistics*, 11(2):416–431, 1983.
- [16] Eric Saund. A multiple cause mixture model for unsupervised learning. *Neural Computation*, 7(1):51–71, 1995.
- [17] Claude E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948.
- [18] Koen Smets and Jilles Vreeken. Slim: Directly mining descriptive patterns. In *Proceedings of the 2012 SIAM International Conference on Data Mining (SDM)*, pages 236–247, 2012.
- [19] Greg Ver Steeg and Aram Galstyan. Discovering structure in high-dimensional data through correlation explanation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, 2014.
- [20] Jilles Vreeken, Matthijs van Leeuwen, and Arno Siebes. Krimp: Mining itemsets that compress. *Data Mining and Knowledge Discovery*, 23(1):169–214, 2011.